

PROJETO: CONEXÕES SEGURAS - Relatório Técnico

Parte 1/3

Disciplina: Tecnologia da Informação e Conectividade

Professor: Wesley Fioreze

Integrantes:

- Bárbara Yasmin Pimenta dos Santos Tobias
- Luana Gabrielle Ferreira Guedes Paes
- Pedro Augusto Lombardi da Costa
- Ruan Monteiro Brito

1. Módulo 1 - Sniffing e Interceptação (Camada 2 e 3)

Conceito

Em condições normais, a NIC rejeita frames cujo endereço MAC de destino não corresponde ao seu. O modo promíscuo desativa esse filtro no driver, enviando todos os frames para o sistema operacional. Isso possibilita a captura passiva do tráfego de outros dispositivos na mesma rede.

Ferramenta e Passo a Passo

- Wireshark 4.6.6 com Npcap - Windows 10, executado como Administrador
- Modo promíscuo ativado: Edit -> Preferences -> Capture
- Captura na interface Wi-Fi; Tráfego gerado com ping google.com e acesso a <http://neverssl.com>
- Filtros aplicados: dns, http, icmp

Resultado

Com filtro HTTP, a requisição ao neverssl.com capturada em texto claro:

```
GET /online HTTP/1.1
Host: wonderfulbeautifulsplendidkiss.neverssl.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)...
Accept-Language: pt-BR,pt;q=0.9 | Referer: http://neverssl.com/
```

Filtro DNS exibiu consultas a google.com, discordapp.com e microsoft.com. Filtro ICMP mostrou Echo Request/Reply com IPs expostos.

Explicação Técnica

O modo promíscuo modifica o driver da NIC a nível de SO. Em redes Wi-Fi, a captura é passiva — sem interação com a vítima. Em redes comutadas seria preciso realizar ARP Poisoning. HTTP revela informações como sistema operacional, navegador, idioma, URLs e cookies — evidenciando a importância do TLS.

2. Módulo 2 - Criptografia e Handshake TLS

Conceito: Criptografia Híbrida

O HTTPS não utiliza RSA/ECC para todo o tráfego por questões de desempenho, uma vez que os algoritmos assimétricos são lentos para lidar com grandes volumes. A resposta é a criptografia híbrida:

Etapa	Algoritmo	Finalidade
Troca de chaves	RSA / ECDHE (assimétrico)	Estabelecer chave de sessão com segurança
Dados	AES-256-GCM (simétrico)	Cifrar o tráfego com alta performance
Integridade	SHA-256 / SHA-384	Garantir que dados não foram alterados

Ferramenta e Passo a Passo

- `pip install trustme` - geração de certificado autoassinado via Python
- `servidor.py`: `SSLContext` + `trustme.CA()` + `HTTPServer` na porta 4443
- Wireshark na interface Loopback (127.0.0.1), filtro: `tls`
- Acesso via Chrome em `https://localhost:4443`

Resultado

Pacote TLS 1.3	Descrição
Client Hello (SNI=localhost)	Navegador anuncia versões e cipher suites

Pacote TLS 1.3	Descrição
Server Hello	Servidor seleciona TLS 1.3
Change Cipher Spec	Transição para comunicação cifrada
Application Data	Dados cifrados — ilegíveis no Wireshark

O navegador apresentou o erro NET::ERR_CERT_AUTHORITY_INVALID devido ao fato de o certificado ser autoassinado. A conexão TLS operou de maneira usual, com os dados sendo transmitidos de forma cifrada.

Explicação Técnica

O ECDHE assegura Perfect Forward Secrecy, utilizando uma chave efêmera distinta para cada sessão. Embora a chave privada possa ser comprometida no futuro, as sessões anteriores não poderão ser decifradas. Depois do Change Cipher Spec, o Wireshark exibe apenas Application Data opaco, em contraste com o HTTP do Módulo 1.

3. Módulo 3 - Segurança na Camada de Aplicação

Conceito: Cookie SessionID

Depois que o login é realizado, o servidor cria um SessionID exclusivo e o envia por meio do Set-Cookie. Em cada requisição, o navegador o envia novamente para identificar o usuário. Como desenvolvedor, as principais preocupações são:

Vulnerabilidade	Risco	Mitigação
Session Hijacking	Cookie capturado em HTTP vira senha	HTTPS + flag Secure
XSS	Script lê cookie via document.cookie	Flag HttpOnly
CSRF	Requisição forjada usa cookie	Flag SameSite=Strict

Vulnerabilidade	Risco	Mitigação
Sem expiração	Sessão aberta indefinidamente	Max-Age + timeout server-side

Ferramenta e Passo a Passo

- pip install flask — aplicação com rotas /login, / e /logout
- Execução em http://localhost:5000 (HTTP puro, sem TLS)
- Wireshark na interface Loopback, filtro: http
- Login realizado; pacote HTTP/1.1 302 FOUND analisado

Resultado

Cookie capturado em texto claro no pacote de resposta ao POST /login:

```
Set-Cookie: session=eyJlc3VhcmlvIjoiUGVkc3Vhcm8ifQ.ahZo4g.0KEfEOy1JS...
HttpOnly; Path=/ ← flag Secure e SameSite AUSENTES
```

HttpOnly presente protege contra XSS. Mas, a ausência de Secure permite que o cookie seja enviado em HTTP sem criptografia — qualquer sniffing captura o SessionID e permite Session Hijacking.

Papel do HTTPS na Proteção de Cookies

O HTTPS cifra toda a camada de transporte com TLS, incluindo os headers onde os cookies trafegam. Com HTTP puro, o SessionID fica exposto na rede. A flag Secure orienta o navegador a enviar o cookie apenas por meio de HTTPS, evitando assim o ataque.

4. Comparativo e Conclusão

HTTP vs HTTPS

Característica	HTTP	HTTPS
Criptografia	Nenhuma	TLS 1.3 · AES-256-GCM

Características	HTTP	HTTPS
Pacotes no Wireshark	Texto claro	Application Data cifrado
Cookie de sessão	Exposto na rede	Protegido pelo TLS
User-Agent / SO	Visível	Oculto
Porta	80 / 5000	443 / 4443
Risco principal	Session Hijacking, Sniffing	Certificado inválido (autoassinado)

Conclusão

As três PoCs mostraram na prática os perigos de protocolos não criptografados e a eficiência do TLS. O Módulo 1 demonstrou que os dados HTTP permanecem expostos de forma passiva. O Módulo 2 demonstrou que o TLS torna os dados ininteligíveis mesmo para quem intercepta os pacotes. O Módulo 3 demonstrou que cookies sem as flags adequadas constituem vetores de ataque concretos.

Conclusão prática: toda aplicação que requer autenticação deve utilizar HTTPS com um certificado válido e configurar os cookies com as propriedades Secure, HttpOnly e SameSite para reduzir ao máximo a superfície de ataque.